Intro + use case diagram

Introduction:

Good morning, we are Group 4 and my name is __ and my team member names are __. Before we go into the live demo, I'll give a quick overview of the main idea behind our product and our use case diagrams. Our product is a website called Jobvago.

<Next slide>

It is designed primarily for job seekers who want to make better career decisions using real market data, relevant upskilling recommendations via SkillsFuture courses, and resume improvement tools. Our product brings all of these into one connected platform so that they can better navigate their way.

<Next slide>

The first main feature is the Top Industries by Job Vacancies view. The system retrieves the latest vacancy data from an API, processes it, and shows the top industries with the highest demand. This gives users a strong starting point, because they are not just exploring blindly but they are looking at where the opportunities actually are in the market.

<Next slide>

From there, the user can then select an industry, and the system retrieves and displays relevant SkillsFuture courses for that area. It shows useful details like the prerequisites, learning outcomes, so the user can immediately see what learning options are available.

<Next slide>

After that, users can filter and sort the courses based on things like impact level, course mode, or language. This is important because different users have different needs. Some may care more about flexibility, some about the highest impact, and some are only fluent in certain languages. So this feature makes the recommendations more practical and personalised, instead of giving everyone a long list of courses that they have to sort themselves and reduces decision overload.

<Next slide>

Another major part of our solution is resume support. Users can upload and manage their resume within the system, and then analyse it to receive a score out of 100, extracted skills, and a breakdown of how their resume performs. This means our platform does not stop at helping users explore opportunities but it also helps them improve their readiness for those opportunities.

<Next slide>

Finally, all of this comes together in the dashboard, which acts as the user's main overview page. It brings together key information such as resume analysis summary, extracted skills, and favourite courses, so the user can keep track of their progress in one place.

So overall, the main strength of our product is that the features are connected into one full platform to aid them in their career switches instead of manually looking at the courses themselves and worrying about which industry to step into. So now that I covered the main functionalities, we will begin the live demo of our product.

live demo

<live demo>

1. Landing Page (~30 sec)
[Open the app. Career landing page loads.]
"This is JobVago, a career guidance app that connects Singapore's job market data with SkillsFuture courses. It pulls data from two government APIs: one for job vacancies by industry, and one for the SkillsFuture course directory.
At the top we have three stats, total vacancies, top 10 industries, and total courses available. Below that, the top 10 industries ranked by vacancy count. The data is cached and refreshes every 24 hours."

2. Industry Selection + Course Browsing (~20 sec)
[Click an industry card]
"Clicking an industry highlights it and loads matching courses below. The matching works through keyword matching, each industry has mapped keywords like 'python', 'cloud', ‘linux’. The system checks course titles against these to return matches.
Each course card shows attributes like price and prerequisites."
[Point to the elements on a card.]

3. Filtering, and Course Detail Modal (~30 sec)
[Select 'High Impact' filter.]
"Users can filter the courses by the attributes of the courses like for example the impact level. I'll filter by High Impact.
"Clicking a course title will reveal the learning outcomes and prerequisites of the course.”
[Close modal.]

4. Register an Account (~30 sec)
[Click “Register” button]
"Some features need an account. The form validates that the email has an '@' symbol, password is at least 6 characters, name is at least 2 characters, and both password fields match. Emails are normalised to lowercase and duplicates are blocked."
[Fill in and submit.]
"After registering, I'm taken to the dashboard."

5. Dashboard (~30 sec)
[Navigate to dashboard.]
"The dashboard shows everything in one place. At the top, favourite count, analysis count, and latest score.
My favourite courses are here, filterable by language.
Below that, my latest resume score with the Impact, Presentation, and Competencies breakdown.
Then my Skills Profile which are skills extracted from the resume, grouped by category.
And finally, Recommended Courses, courses recommended by the system by matching my skills with course prerequisites so it is personalised to my resume."

6. Favourite a Course (~20 sec)
[Go back to career page. Click heart icons on 2–3 courses.]
"Now I can favourite courses by clicking the heart icon. These are saved to my dashboard. If I wasn't logged in, these features would not be available."

7. Resume Upload/Edit + Grading Result (~2min)
[Navigate to resume upload. Upload a PDF.]
"We can upload a resume and the system will analyse the resume out of 100 points."
[Result page loads. Point to the score ring.]
"Here's the result. This badge shows whether AI or rule-based analysis was used.
The grade has three sections, Impact, Presentation, Competencies, similar to VMock.
Right now, the AI is configured to do the same thing as rule based analysis, which is to grade a resume out of 100 points. However, this is just a proof of concept and the AI can be updated in the future to perform much more complex analysis on the resume.
The system also extracts skills from the resume for example, Python, Docker, Linux which will then be used to find courses to recommend to users."

8. Conclusion (~20 sec)
"In summary, JobVago uses government data of top 10 industries by demand and matches them to Skillsfuture training courses with filtering and sorting. Grades and edit resumes, and gives personalised course recommendations based on extracted skills."

<back to slides>

SE practce

Presentation Script: Design & Architecture

Alright, I'll be going through the design of our system, focusing on ECB, MVC, the design patterns we used, and how SOLID principles are applied.

So first, we structure our system using the **ECB model**, which stands for Entity, Control, and Boundary.

The **Boundary** is basically our user interface — the templates that display information to the user.
The **Control** is our controllers, which handle requests and manage the flow of each feature.
And the **Entity** represents our core data, like User, Resume, and Course, which are stored in the database.

So overall, ECB helps us clearly separate user interaction, logic, and data.

Building on that, we use the **MVC architecture**.

The **Model** corresponds to our entities and database layer.
The **View** is the frontend templates.
And the **Controller** handles incoming requests and decides what happens next.

But one thing we did differently is that we added a **service layer** in between the controller and the model.

This is important because it keeps our controllers simple, and all the business logic is placed in the service layer instead of being scattered around. So everything stays more organised and easier to manage.

Next, for **design patterns**, we applied a few key GoF patterns.

First is the **Facade pattern**. Our service classes act as a facade — they provide a simple interface for more complex operations like resume analysis or API calls, so the controllers don't need to deal with the internal details.

We also use the **Factory pattern** when creating structured objects, especially when handling data from external APIs. This helps standardise object creation and reduces tight coupling.

Another one is the **Strategy pattern**, which we use when there are multiple ways to perform a task, like different filtering or analysis logic. This allows us to switch behaviour without changing the overall system structure.

We also make use of Data Transfer Objects, to pass data cleanly between layers, instead of exposing entire models directly.

Finally, our system follows the **SOLID principles**.

For **Single Responsibility**, each component has a clear role — controllers handle requests, services handle logic, and models handle data. Even within the service layer, responsibilities

are further separated by domain, so each service focuses on a single area of functionality instead of becoming overly complex.

For **Open/Closed**, we can extend functionality, like adding new features or logic, without modifying existing code. For example, if we want to introduce a new type of resume analysis, we can add it as a new strategy without modifying the existing logic, which follows the open/closed principle.

For **Liskov Substitution**, interchangeable components, like different strategies, behave consistently. In our strategy-based components, different implementations can be substituted without affecting the correctness of the system, as they all follow the same expected behaviour.”

For **Interface Segregation**, we keep components focused and avoid unnecessary dependencies. Instead of having large, general-purpose classes, our services expose only the specific methods required by each controller, which keeps dependencies minimal.

And for **Dependency Inversion**, higher-level components depend on abstractions, like services, instead of directly depending on low-level implementations. High-level components like controllers do not directly depend on low-level implementations such as database queries or external APIs. Instead, they interact through service abstractions, which reduces coupling and improves flexibility

Overall, SOLID principles are not applied in isolation, but are reflected throughout the layering, service design, and use of patterns. This ensures that the system remains well-structured and avoids common issues like tight coupling and code duplication.

SE practice b

[Slide 1 — ECP]

Now I'll be covering our testing strategy and production practices.

Let's start with how we designed our test cases. When testing the Register and Login functions, you might think — we need to test every possible input. But that's thousands of combinations. Instead, we used Equivalence Class Partitioning.

The idea is simple — group all possible inputs into categories where every value in that group behaves the same way. For example, any password with 6 or more characters will always be accepted. So instead of testing length 6, 7, 8, 9, 10 separately — we just test one of them. That one represents the whole group.

On this slide, you can see we identified 20 partitions across email, password, confirm password, and full name — for both Register and Login. This covers every meaningful scenario without any redundant tests.

[Slide 2 — BVA]

ECP tells us what categories exist. But where exactly within a category should we test? That's where Boundary Value Analysis comes in.

Bugs almost always hide at the edges — not in the middle. So for our minimum password length of 6 characters, we tested exactly three points: length 5, which should be rejected. Length 6, which is exactly on the boundary and should be accepted. And length 7, one above, also accepted.

You can see the actual test code here — BV1, BV2, BV3 — each asserting the correct response. Together with ECP, these two techniques catch the edge case bugs that normal testing completely misses.

[Slide 3 — Basis Path]

For our third technique, we go inside the code. Our resume analyzer has a method called score_specifics — it gives marks based on how many bullet points in your resume contain actual numbers and quantified achievements.

This method has many decision branches — each ratio threshold is a different if-statement. So we drew those branches as a Control Flow Graph, mapping every possible route through the method as nodes and arrows.

Then we calculated the cyclomatic complexity, V(G). In this case, there are 5 decision points, so V(G) equals 6. That tells us mathematically — we need exactly 6 test cases to guarantee every branch is covered at least once. Not 100 tests, not guessing. Just 6, derived from the structure of the code itself.

The example on the right shows Path 2 — 5 bullet points, 4 containing numbers, ratio of 0.8, so the method should return 8 points. The test asserts exactly that.

[Slide 4 — Security]

Moving on to production practices. First, security.

There are three things we did here. First — no hardcoded credentials. The secret key and database address are read from the server's environment variables, never written directly into the code. If they were hardcoded, anyone who sees our GitHub repo sees our credentials.

Second — secure session cookies. HTTPOnly prevents JavaScript from reading the cookie, which blocks XSS attacks. SameSite Lax prevents the cookie from being sent from other websites, blocking CSRF attacks.

Third — file upload protection. Resume uploads are strictly capped at 16MB and only PDF files are accepted. These three practices ensure the app is secure by design, not as an afterthought.

[Slide 5 — Caching]

Next, caching. JobVago relies on government APIs — SkillsFuture and job vacancies from data.gov.sg. If we call these APIs fresh every single time a user loads the page, it's slow and unreliable.

So we implemented a 24-hour file cache. Data is downloaded once, saved locally, and reused for 24 hours. The method shown here, should_refresh, checks two things — does the cached file exist, and is it older than 24 hours? If either is true, it fetches fresh data. Otherwise, it serves from cache.

This makes the app faster, reduces load on government servers, and keeps JobVago working even if the API goes down temporarily.

[Slide 6 — Extensibility]

Finally, extensibility. Instead of scattering keywords, URLs, and settings all over the codebase, everything is centralised.

Industry keywords used to match jobs and courses are all in one dictionary file — adding a new industry tomorrow is a one-line change. And we have separate configurations for development and production environments, so switching between them requires no code changes at all.

This means the codebase is easy to maintain, easy to extend, and ready for future upgrades. That wraps up the Quality and Process section — thank you.

tracebility

[Slide 1](30s)

I'll now present the traceability aspect of our system, JobVego.

In software engineering, traceability ensures that every user requirement is clearly mapped to the system design and implementation. This helps us verify that all features are properly supported and nothing is missing.

In our project, we establish traceability across three main layers:
the use case diagram, the class diagram, and the system functionalities.

Let me illustrate this with a few examples from our system.

[Slide 2](1min50s)

First, for the career exploration feature, the use case includes actions such as *view top industries*, *browse courses*, and *filter courses*.
These are directly supported in our class diagram by classes like Industry, Course, and CourseFilterCriteria.
For example, Industry stores vacancy-related information retrieved from the API, while Course represents available SkillsFuture courses, and CourseFilterCriteria enables personalised filtering.
This shows a clear mapping from user interaction to system structure.

Next, for the resume support feature, the use case includes *upload resume*, *analyse resume*, and *extract skills*.
In our class diagram, these are represented by the Resume and ResumeAnalysis classes.
Resume stores the uploaded file and its content, while ResumeAnalysis stores the results such as scores, extracted skills, and feedback.
This demonstrates how user actions are translated into persistent data structures within the system.

Finally, for the dashboard feature, which integrates multiple parts of the system, the use case includes viewing *analysis results*, *skills*, and *favourite courses*.
These are supported by classes such as ResumeAnalysis and FavoriteCourse, allowing the system to aggregate and display all relevant user information in one place.

Overall, our traceability ensures that every feature described in the use case diagram is consistently reflected in our class diagram and supported by the system design.
This improves the system's completeness, consistency, and maintainability.

Thank you.